

Sección 4 — Juegos

Ruta de autoaprendizaje: grafos y BFS, backtracking y aleatoriedad determinista, desde cero hasta defender el código de esta sección

0. Mapa de la ruta

Fase	Tema	Tiempo orientativo	Resultado
1	Grafos y búsqueda en anchura (BFS)	1-2 semanas	Construir el grafo de Hamming del corpus y explicar por qué BFS da el mínimo
2	Backtracking y problemas de satisfacción de restricciones	1-2 semanas	Trazar a mano la colocación de un crucigrama de 4 palabras, con retroceso
3	Aleatoriedad determinista: LCG, semillas y Fisher–Yates	1 semana	Predecir un tablero de Codenames a partir de su semilla, a mano
4	Implementación y experimentos en el repo	1-2 semanas	Añadir una variante de juego y medirla con <code>simular-juegos</code>

Regla de oro (la misma de las Secciones 2 y 3): cada concepto se aprende **tres veces** — en papel (la definición y la prueba), en la consola (el número) y en el repo (el código que lo usa).

Fase 1 — Grafos y búsqueda en anchura

1.1 El juego más viejo de la teoría de grafos aplicada

La escalera de palabras (*word ladder* / *doublets*) la inventó Lewis Carroll en 1877; Donald Knuth la convirtió en el ejemplo canónico de grafo real en los años 90 con las 5.757 palabras inglesas de 5 letras (el grafo de *words* del Stanford GraphBase). La versión de este repo es la misma idea sobre el corpus alemán de la plataforma: vértices = palabras, aristas = distancia de Hamming 1. Todo lo interesante del juego es una propiedad del grafo: si hay solución (conectividad), cuál es la solución perfecta (distancia) y qué retos existen (distribución de distancias).

1.2 BFS y por qué da el camino mínimo

BFS explora por **niveles**: primero los vecinos a distancia 1, luego los de 2... La invariante — cuando BFS alcanza un nodo, lo hace por un camino de longitud mínima — se demuestra por inducción sobre el nivel, y solo vale en grafos **no ponderados** (con pesos hace falta Dijkstra). Coste $O(V + E)$ con cola; en `src/engine/escalera.js` la cola es la lista «frontera» de cada nivel (`distanciasDesde`) y la reconstrucción del camino guarda el predecesor de cada nodo (`caminoMinimo`).

1.3 La construcción por cubetas comodín

El truco no-obvio del motor: en vez de comparar todas las parejas ($O(n^2 \cdot L)$), cada palabra entra en L cubetas con una posición enmascarada (`hand` → `*and`, `h*nd`, `ha*d`, `han*`) y dos palabras son vecinas ⇔ comparten cubeta. Piénsalo como un hash por «patrón con comodín». Ejercicio de 15 minutos: demuestra las dos direcciones del ⇔ y encuentra qué pasaría si dos longitudes distintas compartieran cubeta (no pueden: la máscara conserva la longitud).

```
node -e "const {construirGrafo, estadisticas} = await import('./src/engine/escalera.js');
const j = JSON.parse(require('fs').readFileSync('src/data/juegos.json', 'utf8'));
console.log(estadisticas(construirGrafo(Object.keys(j.escalera['4']))))"
```

1.4 Qué estudiar y dónde

Recurso	Qué aporta
CLRS «Introduction to Algorithms», cap. de grafos elementales (BFS/DFS)	La prueba formal de que BFS calcula distancias; representaciones de grafos
Knuth, «The Stanford GraphBase» (el grafo de <i>words</i>)	La escalera de palabras como objeto de estudio serio; estadísticas del grafo inglés
Skiena, «The Algorithm Design Manual», cap. de grafos	Criterio práctico: qué representación y qué recorrido usar según el problema

Fase 2 — Backtracking y satisfacción de restricciones

2.1 El patrón general

Backtracking = búsqueda en profundidad sobre soluciones parciales: extender (elegir variable y valor legal), recursar, y si la rama muere, **deshacer** y probar el siguiente valor. Las tres decisiones de diseño que separan un backtracking de juguete de uno útil: (1) **qué poda** hace ilegales las extensiones pronto; (2) **en qué orden** se prueban variables y valores; (3) **cuándo parar** si el peor caso exponencial asoma. En `src/engine/crucigrama.js`: la poda es el anclaje en cruces + reglas de legalidad; el orden es «más cruces primero»; el paro es el presupuesto de 5.000 nodos con degradación $n \rightarrow n-1$.

2.2 El crucigrama como CSP

Variables = palabras a colocar; dominio de cada una = sus colocaciones legales (que cambian con el tablero: por eso el dominio se recalcula al entrar en cada nodo); restricciones = letra igual en el cruce, extremos libres, sin contactos laterales. Ejercicio de 30 minutos: coloca a mano «anna, nein, insel, esel» con las reglas de `puedeColocar` dibujando el tablero en papel cuadriculado, y provoca un retroceso real eligiendo mal la segunda palabra. Después compara con `src/engine/crucigrama.test.js`, que usa exactamente ese pool.

2.3 Presupuestos y anytime

El peor caso del backtracking es exponencial y no se puede eliminar, solo domesticar: un **presupuesto de nodos** convierte el algoritmo en *anytime* (siempre hay respuesta, quizá con $n-1$ palabras). La medición del repo (`npm run simular-juegos`) muestra que con el pool real el presupuesto nunca se activa (100% de éxito en $< 0,5$ ms) — pero el seguro debe existir ANTES de saberlo. Discute: ¿por qué se degrada n en vez de aumentar el presupuesto?

2.4 Qué estudiar y dónde

Recurso	Qué aporta
Russell & Norvig, «Artificial Intelligence: A Modern Approach», cap. de CSP	Formalización de variables/dominios/restricciones, heurísticas de orden (MRV, LCV)
CLRS / Skiena, secciones de backtracking	El patrón extender-recursar-deshacer y sus podas
Knuth, «The Art of Computer Programming» vol. 4B, «Dancing Links»	El siguiente nivel: cobertura exacta para rejillas de crucigrama de periódico

Fase 3 — Aleatoriedad determinista

3.1 El LCG y la semilla como protocolo

Un generador congruencial lineal ($x_{i+1} = (a \cdot x_i + c) \bmod m$) es el PRNG más simple que existe; sus debilidades estadísticas (planos de Marsaglia, período corto en los bits bajos) lo descartan para criptografía o simulación seria, pero aquí la propiedad que importa es otra: **reproducibilidad exacta multiplataforma**. La semilla de Codenames (DP-K9A2) es un protocolo de comunicación de 6 caracteres: dos capitanes en dispositivos distintos derivan el mismo tablero sin servidor. La misma disciplina gobierna las Secciones 2–4: el azar solo entra por la semilla (tests con `toEqual`, retos compartibles, métricas reproducibles).

3.2 Fisher–Yates y el sesgo del barajado ingenuo

Barajar bien es elegir una permutación uniforme: Fisher–Yates lo hace en $O(n)$ intercambiando cada posición con una aleatoria de las restantes (`barajar` en `board.js`). El clásico error — `sort(() => Math.random() - 0.5)` — produce permutaciones sesgadas. Ejercicio de 15 minutos: enumera las 27 trayectorias del barajado ingenuo de `[a, b, c]` y comprueba que las 6 permutaciones no salen equiprobables.

3.3 Una trampa real de este repo

El LCG de `board.js` devuelve `state / (m - 1)` con `state = hash(semilla)...` y el hash de JS puede ser **negativo**: el «uniforme en $[0,1)$ » puede salir negativo o exactamente 1, y Fisher–Yates indexaría fuera del array. La corrección (`crearGeneradorNormalizado`: tomar la parte fraccionaria) vive en `board.js` y la usan gramática, escalera y crucigrama — pero NO se cambió el generador original, porque habría cambiado todos los tableros de Codenames ya compartidos. Moraleja doble: valida el rango de tu PRNG, y trata la función semilla → salida como API pública congelada.

3.4 Qué estudiar y dónde

Recurso	Qué aporta
Knuth, TAOCP vol. 2, «Seminumerical Algorithms», cap. 3	La teoría del LCG: elección de a, c, m ; tests espectrales; por qué los bits bajos son débiles
Fisher–Yates / algoritmo de Durstenfeld (el artículo de Wikipedia es correcto y cita las fuentes)	El barajado uniforme en $O(n)$ y el catálogo de errores clásicos
PCG (<code>pcg-random.org</code>), «PCG: A Family of Simple Fast Space-Efficient...»	Qué usa un PRNG moderno y por qué; contraste instructivo con el LCG

Fase 4 — Implementación y experimentos en el repo

4.1 Recorrido guiado del código

Orden	Archivo	Qué mirar
1	<code>src/engine/board.js</code>	LCG, hash de semilla, Fisher–Yates, composición vocabulario+semilla; el generador normalizado y su porqué
2	<code>src/engine/escalera.js</code> + <code>escalera.test.js</code>	Cubetas comodín, BFS de niveles y de camino, generación de retos a distancia exacta
3	<code>src/engine/crucigrama.js</code> + <code>crucigrama.test.js</code>	Reglas de legalidad, anclaje en cruces, retroceso con deshacer selectivo, numeración
4	<code>pipeline/juegos.py</code>	Qué palabras entran a cada juego y por qué (ASCII, funcionales, frecuencia mínima)
5	<code>simulacion/juegos-stats.mjs</code>	Cómo se mide un motor: mismos módulos que producción, barrido de semillas, salida versionada en docs/

4.2 Experimentos propuestos (de menor a mayor)

(a) Reto diario. Semilla = fecha ISO → todos los usuarios del día juegan la misma escalera. Son ~5 líneas en la UI; discute por qué NO hay que tocar el motor. **(b) Distancia media por longitud.** Amplía juegos-stats.mjs para emitir el histograma completo de distancias y grafica la campana (¿por qué L=4 tiene la distancia media mayor que L=5 en este corpus?). **(c) Heurística MRV.** Cambia el orden de palabras del crucigrama a «menos colocaciones legales primero» y mide con el barrido de semillas si la densidad o el tiempo mejoran. **(d) Umlauts.** Acepta ae/oe/ue como ä/ö/ü en la escalera (normalización en ambos lados) y mide cuánto crece la componente gigante. **(e) Crucigrama personal.** Usa la bolsa de palabras del usuario (Sección 2) como pool — el puente natural entre juegos y repaso espaciado.

4.3 Criterio de «sección defendida»

Sabes defender esta sección cuando puedes: (1) probar la invariante de BFS y explicar las cubetas comodín sin mirar el código; (2) trazar un retroceso real del crucigrama y justificar cada una de las tres reglas de legalidad con un contraejemplo; (3) explicar qué garantiza (y qué NO garantiza) un LCG y por qué aquí basta; y (4) reproducir cualquier cifra de métricas-seccion4.pdf con un comando.